

Network Anomaly Detection - Project Source Code

File: app.py

```
from flask import Flask, request, jsonify
import joblib
import os
import pandas as pd
import io

# Initialize Flask app
app = Flask(__name__)

# Load Machine Learning artifacts
MODEL_PATH = 'models/model.joblib'
SCALER_PATH = 'models/scaler.joblib'
FEATURES_PATH = 'models/features.joblib'

try:
    model = joblib.load(MODEL_PATH)
    scaler = joblib.load(SCALER_PATH)
    feature_columns = joblib.load(FEATURES_PATH)
    print("Successfully loaded ML models and scaler!")
except FileNotFoundError:
    print("Warning: Model files not found. Please ensure they exist in the models/ directory.")
    model = None
    scaler = None
    feature_columns = None

from flask import render_template

# Add a home route that serves the frontend
@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    if 'file' not in request.files:
        return jsonify({'error': 'No file uploaded'}), 400

    file = request.files['file']
    if file.filename == '':
        return jsonify({'error': 'Empty filename'}), 400

    if not file.filename.endswith('.csv'):
        return jsonify({'error': 'Only CSV files are supported'}), 400

    try:
        # Read the uploaded CSV
        stream = io.StringIO(file.stream.read().decode('utf-8'))
        df = pd.read_csv(stream)
```

Network Anomaly Detection - Project Source Code

```
# Strip column names
df.columns = df.columns.str.strip()

# Check if model artifacts are loaded
if not feature_columns:
    return jsonify({'error': 'Model artifacts not loaded on the server'}), 500

# 1. Check for missing columns
missing_cols = [col for col in feature_columns if col not in df.columns]
if missing_cols:
    return jsonify({'error': f'Missing required columns: {missing_cols}'}), 400

# 2. Reorder columns to match training features (this also automatically drops extra
columns)
df = df[feature_columns]

# 3. Convert data to numeric and drop NaN rows
import numpy as np
df.replace([np.inf, -np.inf], np.nan, inplace=True)
for col in df.columns:
    try:
        df[col] = pd.to_numeric(df[col])
    except (ValueError, TypeError):
        pass
df.dropna(inplace=True)

if df.empty:
    return jsonify({'error': 'Uploaded CSV contains no valid data after cleaning'}), 400

# 4. Scale the data using the loaded scaler
X_scaled = scaler.transform(df)

# 5. Predict using the trained model
predictions = model.predict(X_scaled)

# 6. Convert -1 to 'Anomaly' and 1 to 'Normal'
pred_labels = ['Anomaly' if p == -1 else 'Normal' for p in predictions]

# 7. Calculate summary counts
counts = pd.Series(pred_labels).value_counts().to_dict()

# 8. Return results as JSON
return jsonify({
    'total_processed': len(pred_labels),
    'counts': counts,
    'predictions': pred_labels
}), 200

except Exception as e:
```

Network Anomaly Detection - Project Source Code

```
        return jsonify({'error': str(e)}), 500

# Run app in debug mode
if __name__ == '__main__':
    app.run(debug=True)
```

File: templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Network Anomaly Detection</title>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700&display=swap"
rel="stylesheet">
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <div class="container">
        <h1>Network Anomaly Detection</h1>
        <p>Upload a CSV file containing network traffic data to detect potential intrusions and
anomalies using Machine Learning.</p>

        <form id="uploadForm">
            <input type="file" id="fileInput" accept=".csv" required>
            <button type="submit" id="submitBtn">Detect Anomalies</button>
        </form>

        <div id="resultsContainer" style="display: none;">
            <div class="summary-cards">
                <div class="card">
                    <h3>Total Processed</h3>
                    <p id="totalCount">0</p>
                </div>
                <div class="card normal">
                    <h3>Normal Traffic</h3>
                    <p id="normalCount">0</p>
                </div>
                <div class="card anomaly">
                    <h3>Anomalies Detected</h3>
                    <p id="anomalyCount">0</p>
                </div>
            </div>

            <div class="table-wrapper">
                <table id="resultsTable">
                    <thead>
```

Network Anomaly Detection - Project Source Code

```
        <tr>
            <th>Traffic Flow ID</th>
            <th>Status Classification</th>
        </tr>
    </thead>
    <tbody id="tableBody">
        <!-- Results will be injected here via JavaScript -->
    </tbody>
</table>
</div>
</div>

<div id="errorContainer" class="error-msg" style="display: none;"></div>
</div>

<script src="{{ url_for('static', filename='script.js') }}"></script>
</body>
</html>
```

File: static/style.css

```
@import
url('https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;600;800&family=JetBrains+Mono&di
splay=swap');

:root {
    --bg-dark: #050505;
    --card-bg: rgba(20, 20, 25, 0.6);
    --border-color: rgba(255, 255, 255, 0.08);
    --text-primary: #ffffff;
    --text-muted: #a1a1aa;
    --accent-glow: #3b82f6;
    --accent-glow-secondary: #8b5cf6;
    --anomaly-color: #ef4444;
    --anomaly-glow: rgba(239, 68, 68, 0.4);
    --normal-color: #10b981;
    --normal-glow: rgba(16, 185, 129, 0.4);
}

body {
    background-color: var(--bg-dark);
    background-image:
        radial-gradient(circle at 15% 50%, rgba(59, 130, 246, 0.15), transparent 25%),
        radial-gradient(circle at 85% 30%, rgba(139, 92, 246, 0.15), transparent 25%);
    color: var(--text-primary);
    font-family: 'Outfit', sans-serif;
    margin: 0;
    padding: 2rem;
```

Network Anomaly Detection - Project Source Code

```
min-height: 100vh;
display: flex;
justify-content: center;
align-items: flex-start;
}

.container {
  background: var(--card-bg);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);
  border: 1px solid var(--border-color);
  border-radius: 24px;
  padding: 3.5rem;
  width: 100%;
  max-width: 900px;
  box-shadow: 0 0 40px rgba(0, 0, 0, 0.5), inset 0 0 0 1px rgba(255, 255, 255, 0.05);
  margin-top: 3rem;
  animation: scaleIn 0.7s cubic-bezier(0.16, 1, 0.3, 1);
  position: relative;
  overflow: hidden;
}

/* Subtle animated top border glow */
.container::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 2px;
  background: linear-gradient(90deg, transparent, var(--accent-glow),
var(--accent-glow-secondary), transparent);
  background-size: 200% 100%;
  animation: gradientMove 3s linear infinite;
}

@keyframes gradientMove {
  0% { background-position: 100% 0; }
  100% { background-position: -100% 0; }
}

@keyframes scaleIn {
  from { opacity: 0; transform: scale(0.95) translateY(20px); }
  to { opacity: 1; transform: scale(1) translateY(0); }
}

h1 {
  margin-top: 0;
  text-align: center;
  font-size: 3rem;
}
```

Network Anomaly Detection - Project Source Code

```
font-weight: 800;
background: linear-gradient(135deg, #fff 0%, #a1a1aa 100%);
-webkit-background-clip: text;
-webkit-text-fill-color: transparent;
letter-spacing: -1px;
}

p {
  text-align: center;
  color: var(--text-muted);
  margin-bottom: 3rem;
  font-size: 1.1rem;
}

form {
  display: flex;
  gap: 1rem;
  justify-content: center;
  align-items: center;
  margin-bottom: 3rem;
  background: rgba(0,0,0,0.3);
  padding: 1rem;
  border-radius: 16px;
  border: 1px solid var(--border-color);
}

input[type="file"] {
  flex: 1;
  max-width: 400px;
}

input[type="file"]::file-selector-button {
  background: rgba(255, 255, 255, 0.05);
  border: 1px solid rgba(255, 255, 255, 0.1);
  color: white;
  padding: 0.75rem 1.5rem;
  border-radius: 8px;
  font-family: 'Outfit', sans-serif;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.2s;
  margin-right: 1rem;
}

input[type="file"]::file-selector-button:hover {
  background: rgba(255, 255, 255, 0.1);
}

button {
  background: linear-gradient(135deg, var(--accent-glow), var(--accent-glow-secondary));
```

Network Anomaly Detection - Project Source Code

```
color: white;
border: none;
padding: 0.85rem 2rem;
border-radius: 8px;
font-weight: 600;
font-family: 'Outfit', sans-serif;
font-size: 1rem;
cursor: pointer;
transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
box-shadow: 0 0 20px rgba(59, 130, 246, 0.3);
text-transform: uppercase;
letter-spacing: 1px;
}

button:hover {
  transform: translateY(-2px) scale(1.02);
  box-shadow: 0 0 30px rgba(139, 92, 246, 0.5);
}

button:disabled {
  background: #27272a;
  color: #52525b;
  box-shadow: none;
  cursor: not-allowed;
  transform: none;
}

.summary-cards {
  display: flex;
  gap: 1.5rem;
  margin-bottom: 3rem;
  animation: fadeIn 0.8s ease-in;
}

.card {
  flex: 1;
  background: rgba(0, 0, 0, 0.4);
  padding: 2rem;
  border-radius: 20px;
  text-align: center;
  border: 1px solid var(--border-color);
  transition: all 0.4s cubic-bezier(0.175, 0.885, 0.32, 1.275);
  position: relative;
  overflow: hidden;
}

.card::after {
  content: '';
  position: absolute;
  top: 0; left: 0; right: 0; bottom: 0;
```

Network Anomaly Detection - Project Source Code

```
border-radius: 20px;
box-shadow: inset 0 0 0 1px transparent;
transition: all 0.4s ease;
}

.card:hover {
  transform: translateY(-8px);
  background: rgba(20, 20, 25, 0.8);
}

.card h3 {
  margin: 0 0 1rem 0;
  font-size: 0.9rem;
  color: var(--text-muted);
  text-transform: uppercase;
  letter-spacing: 1.5px;
}

.card p {
  margin: 0;
  font-size: 3.5rem;
  font-weight: 800;
  font-family: 'JetBrains Mono', monospace;
}

.card.anomaly:hover::after { box-shadow: inset 0 0 20px var(--anomaly-glow); border: 1px solid
var(--anomaly-color); }
.card.anomaly p { color: var(--anomaly-color); text-shadow: 0 0 20px var(--anomaly-glow); }

.card.normal:hover::after { box-shadow: inset 0 0 20px var(--normal-glow); border: 1px solid
var(--normal-color); }
.card.normal p { color: var(--normal-color); text-shadow: 0 0 20px var(--normal-glow); }

.table-wrapper {
  max-height: 450px;
  overflow-y: auto;
  border-radius: 12px;
  border: 1px solid var(--border-color);
  background: rgba(0, 0, 0, 0.2);
}

.table-wrapper::-webkit-scrollbar { width: 6px; }
.table-wrapper::-webkit-scrollbar-track { background: transparent; }
.table-wrapper::-webkit-scrollbar-thumb { background: rgba(255,255,255,0.1); border-radius: 10px;
}
.table-wrapper::-webkit-scrollbar-thumb:hover { background: rgba(255,255,255,0.2); }

table {
  width: 100%;
  border-collapse: separate;
```


Network Anomaly Detection - Project Source Code

```
border-spacing: 0;
}

th, td {
  padding: 1.25rem 2rem;
  text-align: left;
  border-bottom: 1px solid rgba(255, 255, 255, 0.03);
}

th {
  background: rgba(10, 10, 15, 0.95);
  color: var(--text-muted);
  font-weight: 600;
  text-transform: uppercase;
  font-size: 0.85rem;
  letter-spacing: 1px;
  position: sticky;
  top: 0;
  z-index: 10;
  backdrop-filter: blur(10px);
}

td { font-family: 'JetBrains Mono', monospace; font-size: 0.95rem; }

tr:hover td { background: rgba(255, 255, 255, 0.02); }

.badge {
  padding: 0.4rem 1rem;
  border-radius: 6px;
  font-size: 0.8rem;
  font-weight: 800;
  font-family: 'Outfit', sans-serif;
  letter-spacing: 1px;
  text-transform: uppercase;
}

.badge.normal {
  background: rgba(16, 185, 129, 0.1);
  color: var(--normal-color);
  border: 1px solid rgba(16, 185, 129, 0.2);
}

.badge.anomaly {
  background: rgba(239, 68, 68, 0.1);
  color: var(--anomaly-color);
  border: 1px solid rgba(239, 68, 68, 0.2);
  box-shadow: 0 0 10px var(--anomaly-glow);
}

.error-msg {
```

Network Anomaly Detection - Project Source Code

```
background: rgba(239, 68, 68, 0.05);
border: 1px solid rgba(239, 68, 68, 0.3);
color: #fca5a5;
padding: 1.5rem;
border-radius: 12px;
margin-top: 2rem;
text-align: center;
font-weight: 500;
box-shadow: 0 0 20px rgba(239, 68, 68, 0.1);
animation: shake 0.5s cubic-bezier(.36,.07,.19,.97) both;
}

@keyframes shake {
  10%, 90% { transform: translate3d(-1px, 0, 0); }
  20%, 80% { transform: translate3d(2px, 0, 0); }
  30%, 50%, 70% { transform: translate3d(-4px, 0, 0); }
  40%, 60% { transform: translate3d(4px, 0, 0); }
}
```

File: static/script.js

```
document.getElementById('uploadForm').addEventListener('submit', async (e) => {
  e.preventDefault();

  const fileInput = document.getElementById('fileInput');
  const submitBtn = document.getElementById('submitBtn');
  const resultsContainer = document.getElementById('resultsContainer');
  const errorContainer = document.getElementById('errorContainer');

  if (fileInput.files.length === 0) return;

  // Reset UI state
  errorContainer.style.display = 'none';
  resultsContainer.style.display = 'none';
  submitBtn.textContent = 'Analyzing Traffic...';
  submitBtn.disabled = true;

  const formData = new FormData();
  formData.append('file', fileInput.files[0]);

  try {
    const response = await fetch('/predict', {
      method: 'POST',
      body: formData
    });

    const data = await response.json();
```

Network Anomaly Detection - Project Source Code

```
if (!response.ok) {
    throw new Error(data.error || 'An unexpected error occurred during prediction.');
```

```
}

// Update the summary cards
document.getElementById('totalCount').textContent = data.total_processed;

// Handle counts (in case one category has 0, pandas might omit it from the dict)
document.getElementById('anomalyCount').textContent = data.counts['Anomaly'] || 0;
document.getElementById('normalCount').textContent = data.counts['Normal'] || 0;

// Populate the results table
const tbody = document.getElementById('tableBody');
tbody.innerHTML = ''; // Clear old rows

data.predictions.forEach((prediction, index) => {
    const tr = document.createElement('tr');

    // Row ID
    const tdIndex = document.createElement('td');
    tdIndex.textContent = `Flow #${index + 1}`;

    // Status Badge
    const tdStatus = document.createElement('td');
    const badge = document.createElement('span');
    badge.className = `badge ${prediction.toLowerCase()}`;
    badge.textContent = prediction;
    tdStatus.appendChild(badge);

    tr.appendChild(tdIndex);
    tr.appendChild(tdStatus);
    tbody.appendChild(tr);
});

// Show results with a nice animation
resultsContainer.style.display = 'block';

} catch (err) {
    errorContainer.textContent = err.message;
    errorContainer.style.display = 'block';
} finally {
    submitBtn.textContent = 'Detect Anomalies';
    submitBtn.disabled = false;
}
});
```

File: training.ipynb (Python Code Extracts)

Network Anomaly Detection - Project Source Code

Error reading notebook: Expecting value: line 1 column 1 (char 0)